

Desenvolvimento Mobile

Criando apps com

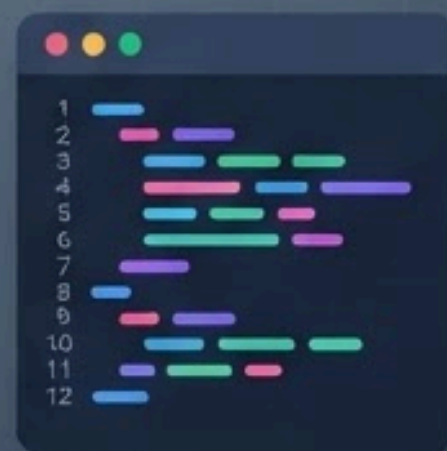
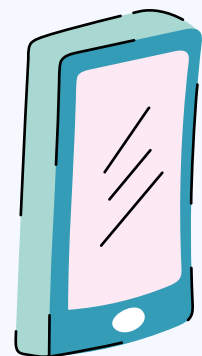


React Native

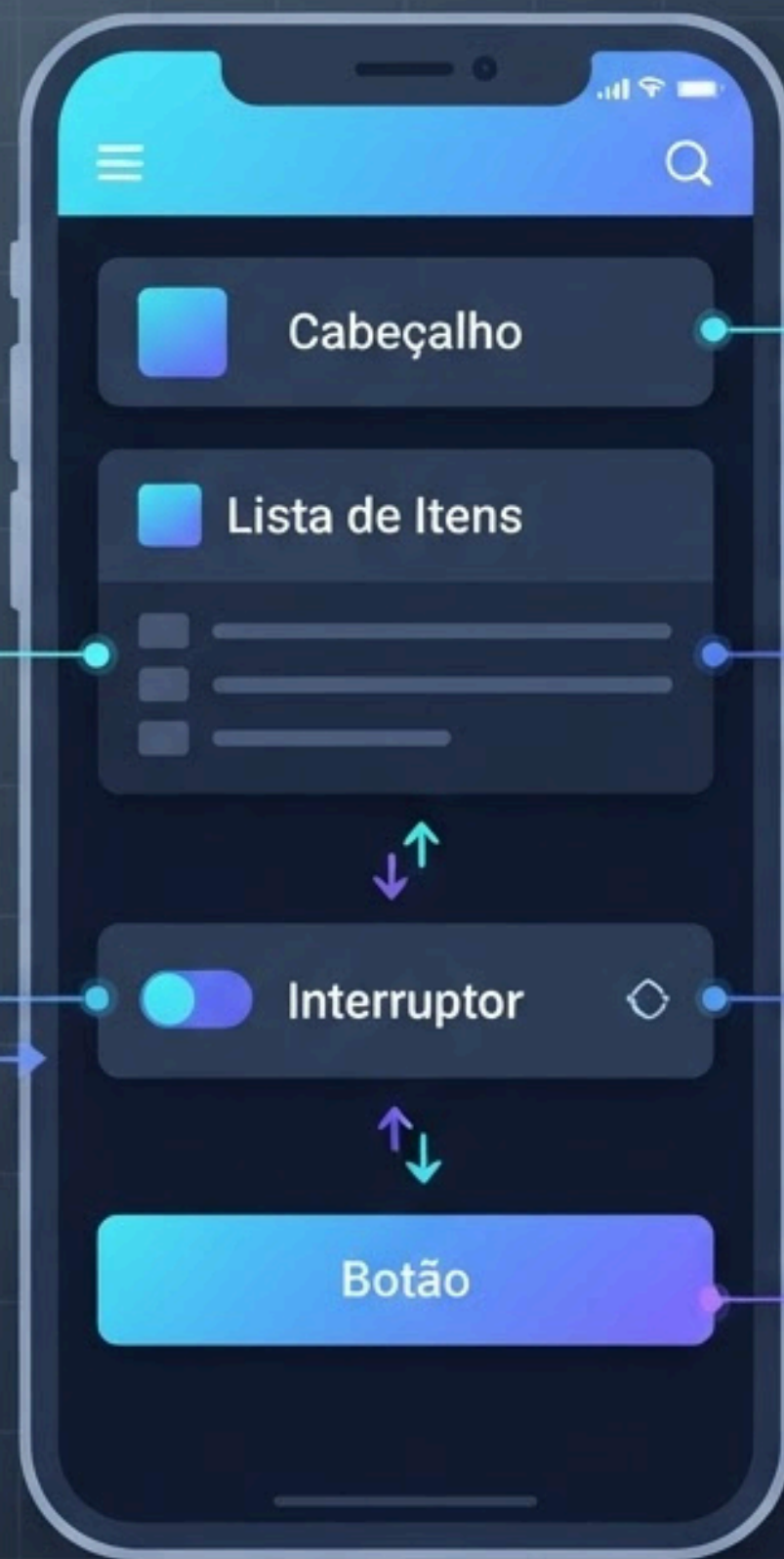


Expo





Editor de Código



Sincronização

• useEffect

• useMemmo

• useState

• useContext

• useMemo

INICIANDO HOOKS

REACT
NATIVE

Os hooks vieram para:

Permitir estado e efeitos em componentes funcionais

Facilitar reuso de lógica (hooks customizados)

Deixar o código mais previsível e isolado

useEffect()

useEffect()

Pense numa lâmpada inteligente:

useState é o interruptor — você liga/desliga, e o estado muda

useEffect é o sensor de movimento — ele reage automaticamente quando algo muda, sem você precisar apertar o interruptor

Sem o **useEffect**, você teria que ligar a lâmpada manualmente em cada lugar onde alguém pode entrar na sala.

Com o **useEffect**, você diz: "quando alguém entrar, acenda" — e pronto, cobre todos os casos.

O que é o useEffect?

O useEffect é um Hook do React que permite executar efeitos colaterais (side effects) em componentes funcionais.

Efeitos colaterais são ações que afetam algo fora do componente, como:

- Buscar dados de uma API
- Manipular o DOM diretamente
- Configurar timers (setTimeout, setInterval)
- Adicionar/remover event listeners
- Sincronizar com sistemas externos

Pense no useEffect como uma forma de dizer ao React: "após renderizar este componente, execute esta ação".

Anatomia do useEffect

```
import { useEffect } from 'react';
```

```
useEffect(() => {
```

```
  // Código do efeito: o que executar
```

```
  return () => {
```

```
    // Função de limpeza (opcional): o que executar antes  
    do próximo efeito ou ao desmontar  
  }
```

```
}, [dependências]) // Array de dependências: controla  
quando o efeito roda
```

Parte	Obrigatória?	Função
Callback	✓ Sim	A função que contém o código a executar
Cleanup	✗ Opcional	Limpa recursos antes de re-executar ou ao desmontar
Dependências	✗ Opcional	Array que define quando o efeito re-executa

Os Três Modos de Uso

```
useEffect(() => {  
  console.log('Executa após TODA renderização');  
});
```

Raramente usado na prática — pode causar loops infinitos se você alterar estado dentro do efeito.

useEffect() Modo 2: Rodar apenas uma vez (array vazio)



```
useEffect(() => {
```

```
  console.log('Executa apenas UMA vez, quando o componente monta')
```

```
  return () => {
```

```
    console.log('Executa quando o componente desmonta')
```

```
  }
```

```
}, []) // array vazio = nenhuma dependência
```

Uso ideal para: buscar dados de API na carga inicial, configurar listeners globais, iniciar timers.

```
useEffect(() => {  
  console.log(`userId mudou para: ${userId}`)  
  buscarDados(userId)  
}, [userId]) // re-executa quando userId muda
```

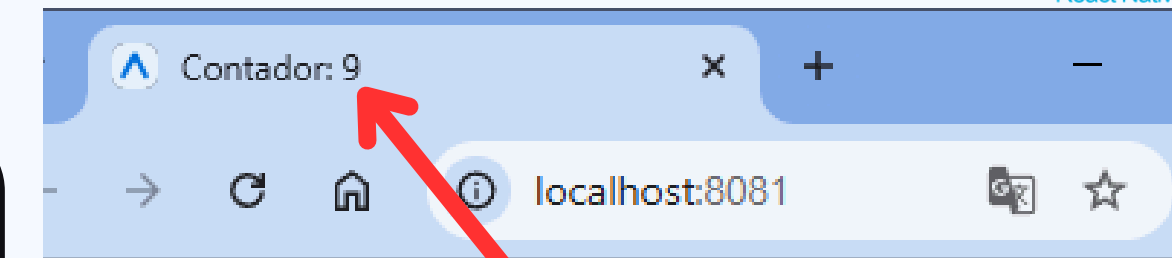
Uso ideal para: reagir a mudanças de props ou estado específicas.

Exemplo 1

```
1 import { useState, useEffect } from 'react'
2 import { View, Text } from 'react-native'
3 export default function Contador1({ titulo = "Contador" }) {
4   //           ↑ valor padrão caso não receba a prop
5   const [contador, setContador] = useState(0);
6   useEffect(() => {
7     document.title = `${titulo}: ${contador}`;
8   }, [contador, titulo]);
9   return (
10     <View style={{flex:1,justifyContent:"center",alignItems:"center"}}>
11       <Text style={{fontSize:30}}>{titulo}</Text>
12       <Text style={{fontSize:30}}>Você clicou {contador} vezes</Text>
13       <button style={{fontSize:30}} onClick={() => setContador(contador + 1)}>
14         Contar
15       </button>
16     </View>
17   )
18 }
```

appUseEffect > App.tsx > ...

```
1 import {View,StyleSheet} from 'react-native'
2 import Contador1 from './src/components/contador1'
3 export default function App() {
4   return (
5     <View style={styles.container}>
6       <Contador1 />
7     </View>
8   )
9 }
10 const styles = StyleSheet.create({
11   container: {
12     flex: 1,
13     backgroundColor: '■ #fff',
14   },
15 })
```



Contador
Você clicou 9 vezes
Contar

Exemplo 2

```

1 import { useState, useEffect } from 'react'
2 import { View, Text, Pressable, StyleSheet } from 'react-native'
3 export default function Contador() {
4   const [contador, setContador] = useState(0)
5   const [corFundo, setCorFundo] = useState('■ #ffffff')
6   // Toda vez que "contador" muda, muda a cor de fundo
7   useEffect(() => {
8     if (contador < 5) {
9       setCorFundo('■ #f31010') // vermelho
10    } else if (contador < 10) {
11      setCorFundo('■ #4809f5') // azul
12    } else {
13      setCorFundo('■ #0ef748') // verde
14    }
15  }, [contador])
16  return (
17    <View style={[styles.container, { backgroundColor: corFundo }]>
18      <Text style={styles.texto}>Você clicou {contador} vezes</Text>
19      <Pressable
20        style={styles.botao}
21        onPress={() => setContador(contador + 1)}>
22        <Text style={styles.texto}>Contar</Text>
23      </Pressable>
24    </View>
25  )}

```

```

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center'},
  botao: {
    alignItems: 'center',
    paddingVertical: 18,
    paddingHorizontal: 40,
    backgroundColor: '■ #f70c0c',
    borderRadius: 10,
    marginTop: 20},
  texto: {
    fontSize: 16,
    fontWeight: '600',
    color: '□ #000000',
    textAlign: 'center'}
})

```

appUseEffect > App.tsx > ...

```
1 import {View,StyleSheet} from 'react-native'
2 import Contador from './src/components/contador'
3 export default function App() {
4   return (
5     <View style={styles.container}>
6       <Contador />
7     </View>
8   )
9 }
10 const styles = StyleSheet.create({
11   container: {
12     flex: 1,
13     backgroundColor: '■ #fff',
14   },
15 })
```

Você clicou 0 vezes

Contar

Você clicou 5 vezes

Contar

Você clicou 10 vezes

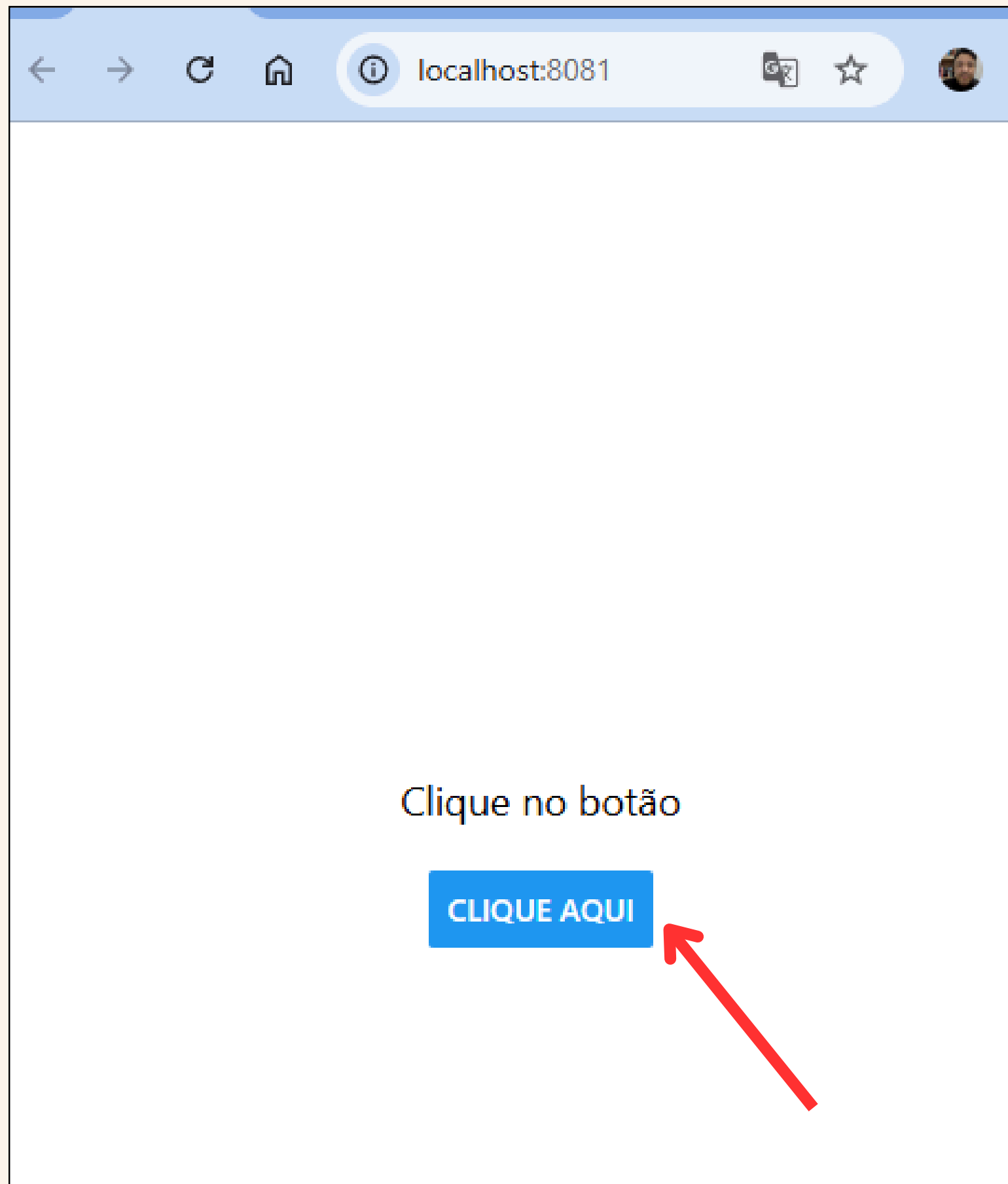
Contar

Exemplo 3

useEffect()

```
1 import { useEffect, useState } from 'react'
2 import { View, Text, Button, StyleSheet, ActivityIndicator } from 'react-native'
3 export default function AnimatedExemplo() {
4   // Estado para controlar se está carregando ou não
5   const [loading, setLoading] = useState<boolean>(false)
6   // useEffect: executa quando o componente monta OU quando 'loading' muda
7   useEffect(() => {
8     // Se loading for true, inicia o carregamento
9     if (loading) {
10       // Aguarda 2 segundos e depois desliga o loading
11       setTimeout(() => {
12         setLoading(false)
13       }, 2000)
14     }
15   }, [loading]) // Dependency array: reexecuta quando 'loading' muda
16   // Função chamada quando o botão é clicado
17   const handlePress = () => {
18     setLoading(true) // Ativa o carregamento
19   }
```

```
21   return (  
22     <View style={styles.container}>  
23       {/* Se loading for true, mostra o spinner. Senão, mostra o texto */}  
24       {loading ? (  
25         <ActivityIndicator size="large" />  
26       ) : (  
27         <Text style={styles.text}>Clique no botão</Text>  
28       )}  
29       {/* Botão que ativa o carregamento */}  
30       <Button  
31         title={loading ? 'Carregando...' : 'Clique aqui'}  
32         onPress={handlePress}  
33       />  
34     </View>  
35   )}  
36 // Estilos do componente  
37 const styles = StyleSheet.create({  
38   container: { flex: 1, justifyContent: 'center',  
39     alignItems: 'center', gap: 20 },  
40   text: { fontSize: 18 },  
41 })
```



Diferença entre useState e useEffect

useState - Apenas MUDA o estado

```
import { useState } from 'react'
import { View, Text, Button, StyleSheet } from 'react-native'

export default function ApenasState() {
  const [count, setCount] = useState<number>(0)

  return (
    <View style={styles.container}>
      <Text style={styles.text}>Cliques: {count}</Text>
      /* useState só muda o valor quando você clica */
      <Button
        title="Clique"
        onPress={() => setCount(count + 1)}
      />
    </View>
  )
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center' },
  text: { fontSize: 20 },
})
```

O que faz: Só muda o número quando você clica. Nada mais.

useEffect - REAGE quando algo muda

```
import { useEffect, useState } from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';
export default function ComEffect() {
  const [count, setCount] = useState<number>(0);
  // useEffect REAGE quando count muda
  useEffect(() => {
    console.log(' AVISO: Você clicou! Count agora é:', count);
    // Aqui você pode fazer coisas AUTOMÁTICAS
    // Como: buscar dados, salvar no banco, enviar para servidor, etc
  }, [count]);
  return (
    <View style={styles.container}>
      <Text style={styles.text}>Cliques: {count}</Text>
      <Button
        title="Clique"
        onPress={() => setCount(count + 1)}
      />
    </View>
  )
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center' },
  text: { fontSize: 20 },
})
```

O que faz: Quando você clica, useState muda o número. Automaticamente, useEffect reage a essa mudança e executa o código dentro dele.

Exemplo Real: Salvar no Banco de Dados

```
import { useEffect, useState } from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';

export default function SalvarDados() {
  const [nome, setNome] = useState<string>('');

  // useEffect REAGE quando nome muda
  useEffect(() => {
    if (nome) {
      // Aqui você AUTOMATICAMENTE salva no banco
      console.log('💾 Salvando no banco:', nome);
      // salvarNoBanco(nome);
    }
  }, [nome]);

  return (
    <View style={styles.container}>
      <Text>Nome: {nome}</Text>
      {/* useState só muda o valor */}
      <Button title="Digitar 'João'" onPress={() => setNome('João')} />
      <Button title="Digitar 'Maria'" onPress={() => setNome('Maria')} />
    </View>
  )
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center', gap: 10 }
})
```

O que acontece:

Você clica no botão → useState muda nome para "João"

Automaticamente → useEffect reage e salva no banco